



B Submission Kit Architecture

Project: B Submission Kit

Repository: <https://github.com/TProver/B-Submission-Kit>

Audience: maintainers, contributors, anyone considering extending or auditing the tool.

Provided by: [CLEARSY](https://clearsy.com) (Safety Solutions Designer).

Atelier B: <https://www.atelierb.eu>

This document describes how BSK is put together, why its pieces are shaped the way they are, and what you can change without rewriting from scratch.

1. Context and goals

A teacher wants to verify B-method submissions from a cohort of ~20-50 students, in real time, during a teaching session, with minimal interaction. Students submit from inside Atelier B; the teacher watches a dashboard and at the end produces an archival PDF report.

Constraints that shaped the design:

- **Run on a single classroom PC.** No cloud, no databases. SQLite was deliberately rejected as overkill; state lives in a JSON snapshot.

- **Student side must be near-zero install.** A `.etool` plug-in plus stdlib-only Python (~120 lines). No `pip install` on student machines.
- **Atelier B + ProB are the verification engines.** They have their own quirks (Windows path conventions, SICStus warm-up cost, weakly-typed PROPERTIES rejection) that the orchestrator must absorb.
- **Submissions arrive concurrently.** Students hit Submit independently; the server must accept them without blocking and process them serially without races.
- **A jealous student must not be able to impersonate another.** Resolved with first-come-first-serve name claiming and a server-issued secret per Atelier B install.

2. Component overview



```

|
| state.py      (JSON)
| reports/      (Edge PDF)
| logs/         (rotating)
+-----+
|
v
Browser dashboard
(HTML, polls every 2 s)

```

Each box's purpose:

Component	Role	Source
BSK plug-in	Two <code>.etool</code> menu entries inside Atelier B that shell out to a Python client	<code>plugin/</code>
bsk_client.py	Connects to the server, zips the project, posts the upload; pure stdlib	<code>plugin/bsk_client.py</code>
bsk_run.cmd	Wrapper that finds a Python interpreter on the student machine	<code>plugin/bsk_run.cmd</code>
server.py	FastAPI app: HTTP endpoints, queue worker, atomic-swap pipeline	<code>receiver/server.py</code>
state.py	In-memory state with JSON snapshot persistence; auth helpers	<code>receiver/state.py</code>
verify.py	Verification pipeline: bbatch driver, output parser, ProB scenario runner	<code>receiver/verify.py</code>
dashboard.html	Single-page browser UI; polls <code>/api/status</code> every 2 s	<code>receiver/dashboard.html</code>
scenarios	Teacher-authored per-project animation scenarios	<code>receiver/scenarios/<Project>/*.scenario</code>

External dependencies:

- **Atelier B** (Community Edition 24.04.2+): provides `bbatch.exe` for the typecheck/POG/prove pipeline.
- **ProB**: provides `probccli.exe` for trace-replay animation.
- **Microsoft Edge**: used in headless mode by `_html_to_pdf` for PDF report rendering.

3. Data flows

3.1 Connect

student plug-in	server	state.py
--POST /api/connect--->		
{student_id,		
hostname,		
session_token?}		
	--claim_name(sid, host, t)->	
		(no entry yet)
		-> generate 32-char
		token
		-> bind sid -> token
		-> persist _state.json
	<--("claimed", token)-----	
<--200 {ok: true,-----		
session_token}		
(write token to		
%APPDATA%\BSK...		
config.json)		

Subsequent Connects from same plug-in send the stored token:

--POST /api/connect---->		
{sid, host,		
session_token=T1}	--claim_name(sid, host, T1)>	
		(existing match)
	<--("rebound", T1)-----	
<--200 {ok, T1}-----		

A different machine claiming the same name without the token:

--POST /api/connect---->		
{sid, host=other}	--claim_name(sid, ...)----->	
		(token mismatch)
	<--("conflict", "")-----	
<--409 {error: name----		
already taken}		

3.2 Submit

The hot path. Two design decisions matter most: (a) the submit handler MUST NOT touch any directory the worker is reading; (b) the worker handles `latest_<project>/` swap as a single atomic operation at the end, with `retry-on-Windows-permission-error`.

student	plug-in	server (HTTP)	asyncio	queue	worker	verify.py
	--POST /api/submit-->					
	multipart:					
	student_id	--check_token()----->				
	project_name	401 if mismatch				
	session_token					
	file=zip					
		--write zip to				
		submissions/<sid>/				
		_incoming/<sid				
		-<proj>-<ts><ms>Z.zip				
		--state.record(...)				
		--queue.put(...)----->				
	<--200 {ok, sub_id}--					
				--get()----->		
					--mkdir _work_<id>/	
					/verification/	
					--shutil.move	
					_incoming -> _work	
					--verify(...)---->	
						--unzi
						--bbat
						(m t
						--per-
						prob
						(up
					<---Result-----	
					--write per-stage logs	
					--_retry_rename:	
					latest_<project>	
					-> _swap_<id>	
					work<id>	
					-> latest_<project>	
					rmtree _swap	
					--state.update(...)	
					--queue.task_done	

Key invariant: the **submit handler's only filesystem write is into** `_incoming/<unique-id>.zip` , which is never touched by the worker for any other submission. The submit ID includes the project name and a

millisecond suffix so two submissions from the same student in the same UTC second cannot collide.

3.3 Dashboard polling

The dashboard is purely client-side and stateless. JS polls `GET /api/status` every 2000 ms. The response is a JSON snapshot built by `state.snapshot()`. The dashboard re-renders the table with the chosen sort mode (Time / Student / Project, persisted in `localStorage`).

3.4 Report generation

`POST /api/report` with no body. The handler:

1. Gets the current `state.snapshot()`.
2. Calls `_build_report_html()` which walks each (student, project) row, reads every `<stage>.log` and `animate_<scenario>.log` file for that submission, and embeds them as `<pre>` blocks alongside the per-(student × project) summary table.
3. Writes the HTML to `reports/<filename>.html`.
4. Spawns `msedge --headless=new --print-to-pdf=...` to convert to PDF.
5. Returns `{ok, pdf_url, html_url}`.

The dashboard's *Generate PDF report* button calls this endpoint and opens the returned `pdf_url` in a new tab.

4. State model

4.1 Per-student structure

```
{
  "alice_dupont": {
    "id": "alice_dupont",
    "secret": "Xb7Yz9-...32-char...",
    "connected_at": "2026-05-02T09:02:04Z",
    "last_activity": "2026-05-02T09:14:31Z",
    "hostname": "STUDENT-PC-42",
    "submissions": {
      "Airlock": {
        "submission_id": "alice_dupont-Airlock-20260502T090205456Z",
        "timestamp": "2026-05-02T09:02:05Z",
        "size_bytes": 1301,
        "status": "partial",
        "stages": {
          "typecheck": "ok",
          "pog": "ok",
```

```

    "prove": "ok",
    "animate": "partial"
  },
  "scenarios": {
    "animate": [
      {"name": "01_smoke_test", "verdict": "ok", "note": ""},
      {"name": "02_normal_use", "verdict": "ok", "note": ""},
      {"name": "05_bad_op", "verdict": "fail",
        "note": "ko at line 3 (this_op_does_not_exist)"}
    ]
  },
  "summary": "typecheck:ok, pog:ok, prove:ok, animate:partial (4/5 passed; failed: 05_ba
},
  "Tunnel": { ... }
}
}
}

```

Each `(student_id, project_name)` pair is its own dashboard row and has its own `submissions/<sid>/latest_<project_name>/` directory on disk.

4.2 Persistence

`state.py` writes the in-memory dict to `submissions/_state.json` after every mutation, atomically (write to `_state.json.tmp` then `os.replace`). On startup the server reads it back. No SQLite, no migrations.

`snapshot()` is the only data surface exposed to the dashboard. It deliberately strips the `secret` field and replaces it with a boolean `claimed` so the dashboard never has access to credentials.

4.3 On-disk layout per submission

```

submissions/
├─ <sid>/
│   ├── _incoming/                # submit handler drops zips here
│   │   └─ <submission_id>.zip
│   ├── _work_<submission_id>/    # worker's per-submission staging
│   │   ├── submission.zip
│   │   └─ verification/
│   │       ├── bbatch_commands.txt
│   │       ├── extracted/        # unzipped student project
│   │       ├── summary.log       # full bbatch transcript
│   │       ├── typecheck.log
│   │       ├── pog.log
│   │       └─ prove.log

```

```

|       ├── animate.log           # aggregate animate log
|       ├── animate_01_smoke.log  # per-scenario logs
|       └── ...
└── latest_<project>/            # atomic-rename target after verify
    ├── submission.zip
    └── verification/...

```

Only `latest_<project>/` is read by the dashboard's log endpoints; everything under `_work_*/` is internal staging that gets renamed in place.

5. Concurrency

5.1 Submit handler vs worker

The submit handler runs on FastAPI's asyncio event loop. The verification worker is a background `asyncio.create_task` started in `@app.on_event("startup")`. Both share the same loop and the same `state` object.

Earliest design tried to enforce "keep only latest" by `shutil.rmtree(latest_dir)` inside the submit handler. On Windows this raced with the worker writing into `latest_<project>/verification/` and produced HTTP 500 file-in-use errors. The fix: **the submit handler never touches any path outside `_incoming/`**. Every mutation of `latest_<project>/` happens inside the worker.

5.2 Single worker

The worker drains the queue serially. One verification at a time. For 50 students × 2 projects × ~15 s per verification ≈ 25 min of total queue time; teachers can watch progress on the dashboard during the session and ask anyone whose verification has not yet completed to wait.

A multi-worker design was rejected because:

- bbatch + ProB are CPU-heavy and serialise well on a single workstation.
- Atelier B's workspace concept doesn't tolerate concurrent access to the same project; each submission already gets a uniquely-named project in `server_workspace/`, but the workspace as a whole is not designed for parallel modification.
- Race surface explosion for marginal speedup.

5.3 Retry-rename pattern

The atomic swap at the end of verification (`staging.rename(latest_dir)`) intermittently fails on Windows with `PermissionError: [WinError 5]` because antivirus / Search Indexer can hold transient handles on the just-written files. The `_retry_rename` helper retries 8 times with exponential backoff (250 ms × 2^k, capped). This is the same pattern git uses internally.

6. Verification pipeline

6.1 bbatch driver

`verify.py` builds a single bbatch command file:

```
crp <bsk_project> <bdp_dir> <lang_dir> SOFTWARE
op <bsk_project>
af <component_1>
af <component_2>
...
m t
m po 0
m pr 0
sg
clp
q
```

Then runs `bbatch.exe -i=<commands.txt>` and **captures stdout** (NOT `-f=`, since bbatch does not flush `-f=` on abort). The resulting transcript is `summary.log`.

`m t`, `m po 0`, `m pr 0` are bbatch's "make-all" forms that operate on every component in the open project. `sg` prints the global status table at the end, which is the canonical source for per-component verdicts.

6.2 Output parser

`_parse_output()` is structured to be **robust to bbatch aborting mid-script**. On a clean run it looks for the `Project status` table and reads its `TOTAL` row to determine `typecheck:ok|fail`, `pog:ok|fail`, and computes the prove verdict from `nUn` (unproved POs).

When `Interpretation aborted at line N` appears, the parser falls back to per-stage content markers:

- `typecheck`: counts `Type Checking machine|implementation|refinement` headers vs `End of Type checking` markers; passing if balanced.
- `pog`: `Pog generation` seen and `PO Generate error` absent → ok.
- `prove`: `Proving X` or `Proof pass` seen → ok, otherwise `skipped`.

This way a model that typechecks but fails POG (e.g. due to bbatch's `succ()` limitation) shows `typecheck:ok`, `pog:fail`, `prove:skipped` rather than blanket-failing every stage.

6.3 ProB animation stage

Each `.scenario` file is converted to a Prolog-format ProB trace file at `_scenario_to_trace()` time and replayed via `probccli <impl> -trace_replay prolog <trace>`. Per-scenario timeout is 30 s (covers SICStus

first-load).

The animate stage uses three-tier classification:

- All scenarios passed → `animate: ok`
- All scenarios failed → `animate: fail`
- Mix → `animate: partial`

If trace replay fails, `_probcli_has_errors` extracts the failing operation name from probcli's `OPERATION: <name> ERROR CONTEXT` line and `_animate` maps it back to the teacher's 1-indexed scenario file line number, surfacing `ko at line N (op_name)` in the verdict.

The animate stage is skipped when prove did not succeed; running ProB on an unloaded model produces noise, not signal.

7. Security

7.1 Threat model

- Network is the classroom LAN. We assume no external attackers.
- Students can read each other's dashboard rows. That is acceptable because results are visible to all anyway.
- The threat we defend against is **a student impersonating another to lower their grade**. Specifically: student A submits a deliberately-broken project under student B's name to make B's row look bad on the dashboard.

7.2 Dynamic name claim

The auth scheme is **first-come-first-serve name claiming with a server-issued secret per Atelier B installation**. There is no pre-distributed credential and no teacher setup beyond starting the server.

- On the first `POST /api/connect` for a name, the server generates a 32-character URL-safe random token (`secrets.token_urlsafe(24)`), binds the name to it, and returns the token to the client.
- The plug-in stores the token in `%APPDATA%\BSKSubmissionKit\config.json` (`~/.config/bsksubmissionkit/config.json` on Linux) and sends it back on every later Connect and Submit.
- A different machine attempting to claim the same name without the token receives HTTP 409 with a clear message.
- A Submit that omits or mismatches the token receives HTTP 401 with a "Re-run Connect" message.

The plug-in installation **is** the identity. There is no password, no SSO, no external auth dependency.

7.3 What the dashboard exposes

`state.snapshot()` strips the `secret` field from every student record before serialising. The dashboard receives only `{id, hostname, project_name, last_activity, claimed (boolean), liveness, last_submission}`. There is no API surface that exposes secrets, and there is no route that returns the raw `_state.json`.

Per-stage / per-scenario log endpoints (`/api/student/{sid}/log?stage=...&project=...`) are public but only return the contents of plain text log files; they do not include any auth material.

7.4 Limitations

- The server has no transport-level encryption (HTTP, not HTTPS). For the classroom-LAN threat model this is fine; for any deployment beyond a trusted LAN, terminate TLS at a reverse proxy (Caddy / nginx).
- A local attacker on the teacher PC with read access to `submissions/_state.json` can read all student secrets. The file is not signed or encrypted.
- A student who manages to read another student's `%APPDATA%\BSKSubmissionKit\config.json` (e.g. via shared logon on the same machine) can impersonate them. Standard OS file permissions are the defence.
- HTTP 409 / 401 messages disclose whether a name is claimed; this is intentional (so legitimate students know to pick a different name), not a leak.

8. Configuration surface

8.1 Environment variables

All paths to external tools are overrideable so the server can run on machines with non-default install locations:

Var	Used by	Default
<code>BSK_BBATCH</code>	<code>verify.py</code>	<code>C:\Program Files\Atelier B Community Edition 24.04.2</code> <code>24.04.2\bin\bbatch.exe</code> (Windows) / <code>bbatch</code> (Linux)
<code>BSK_PROBCLI</code>	<code>verify.py</code>	<code>C:\Tools\ProB\probcli.exe</code> (Windows) / <code>probcli</code> (Linux)
<code>BSK_EDGE</code>	<code>server.py</code>	<code>C:\Program Files</code> <code>(x86)\Microsoft\Edge\Application\msedge.exe</code>
<code>BSK_PYTHON</code>	<code>bsk_run.cmd</code> (student side)	discovered via PATH
<code>BSK_WORKSPACE</code>	<code>verify.py</code>	<code><project>/server_workspace</code>
<code>BSK_SCENARIOS</code>	<code>verify.py</code>	<code><project>/receiver/scenarios</code>

8.2 Scenarios layout

Per-project, in `receiver/scenarios/<ProjectName>/`. Up to 5 `.scenario` files per project (extras are ignored). Filename stem (without extension) is the chip name on the dashboard.

A legacy single-file layout (`receiver/scenarios/<stem>.scenario`) is still supported and resolves to one anonymous "default" scenario, but only if no per-project directory exists.

8.3 verify.py as teacher hook

`verify.py` is intentionally written as a **teacher-replaceable file**. The bbatch command list (`m t` , `m po 0` , `m pr 0` , `sg`), the verdict computation (`_overall_verdict`), the scenario discovery, and the success criteria are all in this one file. A teacher can:

- swap in different bbatch commands per session (e.g. add `b0c` or `b2c`),
- tighten the prove verdict to require `n_un == 0` ,
- restrict component discovery to a specific main machine,
- add an extra stage (e.g. C-code compilation and harness execution).

A future enhancement could move per-session config into a sibling `verify_config.py` so multiple verification flavours can coexist without git-conflicting on `verify.py` itself; for now, just edit `verify.py` and restart the server.

9. Extension points and known limits

Limit	Why it exists	When to revisit
Single asyncio worker	Atelier B workspace concurrency model	Cohort > 50 students
State in JSON snapshot	Avoids any database dependency	When concurrent state writes appear (multi-worker)
HTTP only, no TLS	Trusted classroom LAN	Any deployment beyond a trusted LAN
5 scenarios per project	Caps animate-stage worst-case time at $5 \times 30 \text{ s} = 150 \text{ s}$	Asymmetric cohorts where some projects need more
Single animation target per submission	Each submission has one main implementation	Multi-implementation projects
Plug-in is Windows-first	Atelier B + Tk on Windows is the primary classroom platform	Linux teaching labs

The codebase favours obviousness and traceability over abstraction. Functions are kept short and named after what they actually do; the file count is small enough that ripgrep is the right way to navigate. There is no plugin architecture beyond `verify.py`, no DI container, no ORM, just FastAPI handlers wrapping straightforward Python and subprocess calls.